Lecture 9: Variables , Constants and Typecasting in C

- By the end of this lesson, you will be able to:
  - ✍ Declare variables and constants in C
  - ✍ Apply proper naming conventions when creating variables.
  - ✍ Describe the different types of constants in C.
  - ✍ Explain the difference between variables and constants and their roles in program design.
  - ✍ Describe and perform type conversion in C, including implicit and explicit (type casting) conversions.

Lecture 9: Variables , Constants and Typecasting in C

Variables in C

A named memory location whose value can change during program execution.

Syntax for variable declarations

<type><name>;

- Where type is a primary type e.g. double, char, float and int.
- Name is any valid identifier

Rules followed when declaring variables.

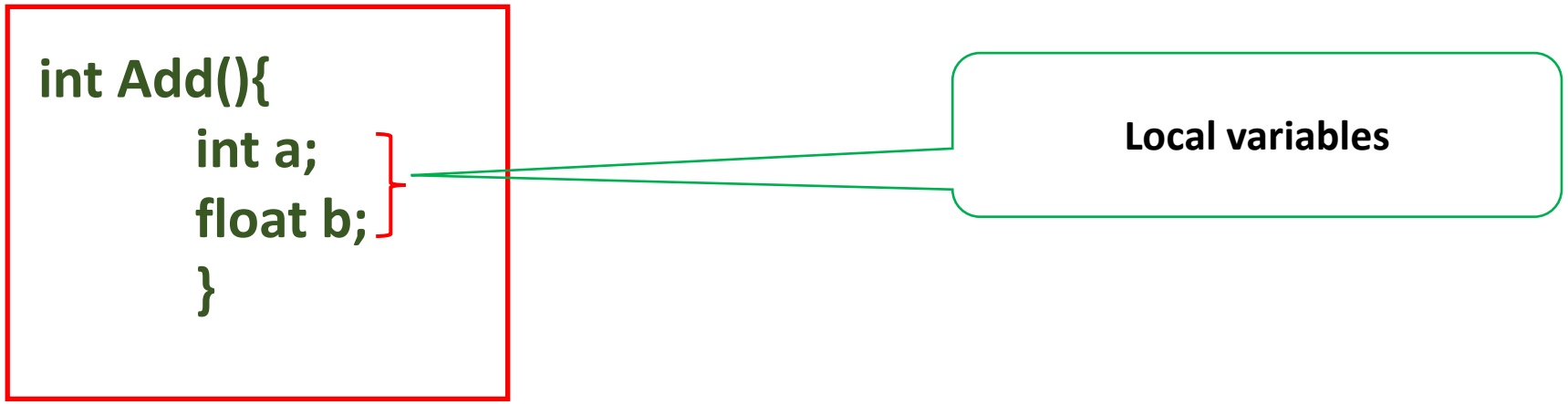
- Use letters, numbers and or underscore.
- Begin with a letter.
- Must be meaningful and easy to remember.
- Do not use reserved keywords.
- C is case sensitive

Lecture 9: Variables , Constants and Typecasting in C

Variables in C

We can classify variables based on their scope:

a) **Local variables**= Only visible within the function or block of declaration.



Lecture 9: Variables , Constants and Typecasting in C

Variables in C

We can classify variables based on their scope:

b) Global variables= Visible anywhere within the program.

```
#include<stdio.h>
int x;
float y;
        void SomeFunction(){
      //do task here
}
```

Global variables

Lecture 9: Variables , Constants and Typecasting in C

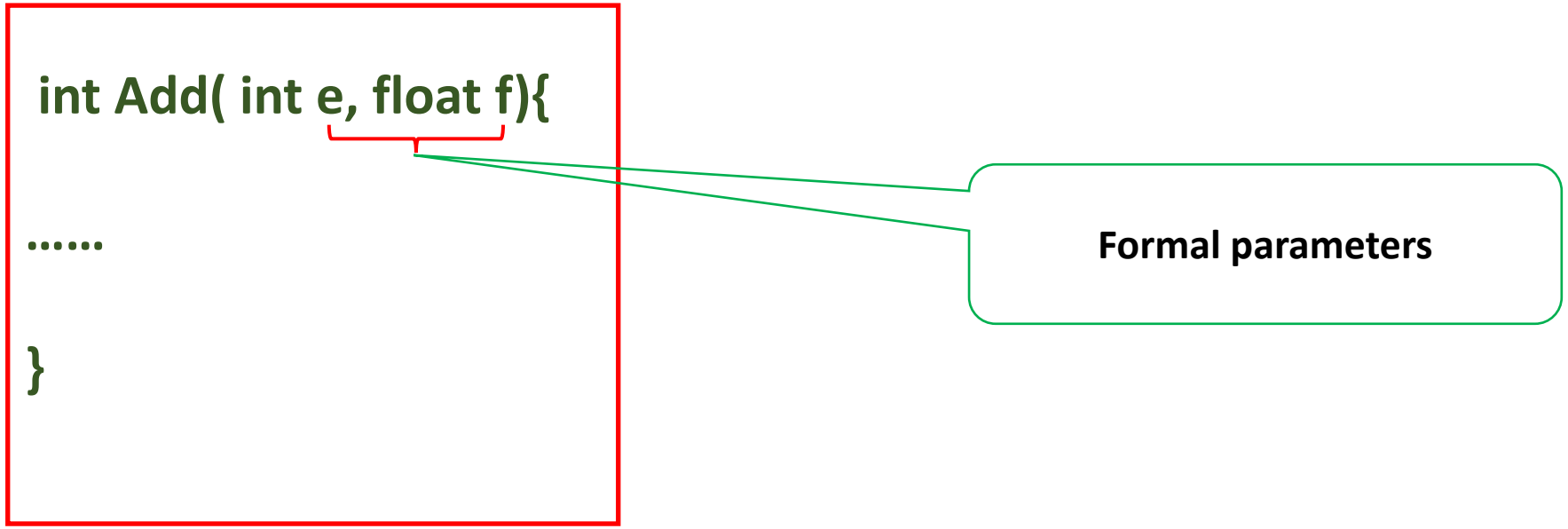
Variables in C

We can classify variables based on their scope:

c) Formal parameters= Appear as placeholders in function header.

Variable(s) declared in the **function header**

They act as placeholders that receive values when the function is called.



Lecture 9: Variables , Constants and Typecasting in C

Variables in C

Why declare variables?

- The compiler will know **type** of value to store.
- The compiler know the **name** of the variable.
- The compiler can allocate adequate **memory size**.
- The compiler can know range values.
- The compiler can determine the **address** of the variable

Variables in C

```c
#include<stdio.h>
int sum; //global variables
void Add(int x,int y) //formal parameters x and y
{
sum=x+y;
printf("The sum of %d and %d is %d\n",x,y,sum);
}
int main()
{
    int a, b;  //local variables a and b
    printf("Enter first and second numbers\n");
    scanf("%d%d",&a,&b);
    Add(a,b); //Calling a function
    return 0;
}
```

Lecture 9: Variables , Constants and Typecasting in C

Variables in C

Activity 1:

Some programming languages are considered to be loosely typed while others are considered to be strongly typed. Discuss the difference and give examples.

Activity 2

For each of the following cases identify error in the variable declaration:-

i. char register;
ii. int volatile
iii. double num;

Lecture 9: Variables , Constants and Typecasting in C

Constants in C

A constant is a fixed value in a program that does not change during execution

Types of constants in C

a) **Literal constants:** Can be literal(typed directly into the code).
   i. Numeral literals:
      - Integer literals e.g. 0,2,3,4,
      - Real number literals e.g. 0.0045,5.6
      - Hexadecimal literals e.g. 0x00BC9F
   ii. Non-numeric literals
      - String literals e.g. “Hello”
      - Character literals e.g. ‘B’
      - Format specifiers e.g. %d,%c,%s,%f
      - Escape sequence characters e.g. \t,\n,,\a

b) **Symbolic constants:** Can be referred to using the symbol.
   - A named memory location whose value does not change at run time.
   - **#define** directive e.g. #define PI 3.142
   - **const** keyword e.g. const float PI=3.142;

Lecture 9: Variables , Constants and Typecasting in C

Constants in C

- Symbolic constant PI declared using #define:

```
#include<stdio.h>
#define PI 3.142  //symbolic constant
float area;  //global variable area
void Calculate(float r) //formal parameter r
{
area=PI*r*r;
printf("The area of a circle of radius %f is %f\n",r,area);
}
int main()
{
    float radius;  //local variable radius
    printf("Enter the radius\n");
    scanf("%f",&radius);
    Calculate(radius); //Calling a function
    return 0;
}
```

Lecture 9: Variables , Constants and Typecasting in C

Constants in C

- Symbolic constant PI using **const** keyword:

```
#include<stdio.h>
const float PI=3.142; //symbolic constant PI
float area;  //global variable area
void Calculate(float r) //formal parameter r
{
area=PI*r*r;
printf("The area of a circle of radius %f is %f\n",r,area);
}
int main()
{
    float radius;  //local variable radius
    printf("Enter the radius\n");
    scanf("%f",&radius);
    Calculate(radius); //Calling a function
    return 0;
}
```

Variables and constants in C

Constants in C

a) **Format specifiers=** Always preceded by %.

Format specifier	Type	Keyword
%c	character	char
%d	Integer	int
%e	Exponents	
%f	Floating points	float
%lf	Double float	double
%p	Pointers/hexadecimals	
%s	String/set of characters	

Constants in C

b) Escape sequence characters= Always preceded by \.

Escape sequence	Meaning
\n	New line
\t	Horizontal tab
\v	Vertical tab

Lecture 9: Variables , Constants and Typecasting in C

Constants in C

Symbolic constants-Referred to using a name or symbol

Const float C=3.0E+08;

Constant	value	C notation
Mass of electron	9.1094×10^{-31}	9.1094E-31
Pi	3.142	3.142
Electron charge	1.6×10^{-19}	1.6E-19
Speed of light	3.0×10^{8}	3.0E+08

Lecture 9: Variables , Constants and Typecasting in C

Constants in C

Activity:

List any three constants you know in the following fields:

a. Finance
b. Engineering
c. Mathematics
d. Computing

Type casting in C

Definition: The conversion of a value from one data type to another.

Type Casting can be classified into **implicit** and **explicit** type casting

(a) Implicit Type Casting

- Done **automatically** by the compiler.
- Occurs in **mixed-type** expressions.
- Smaller type is **promoted** to a larger type.

Example:

int + float ->float

Promotion order:

char ->short -> int -> long -> float

Type casting C

(b) Explicit Type Casting

- Done **manually** by the programmer
- Uses syntax: **(type) <expression>**

Used to:

- **Control precision**: ensure desired decimal accuracy. E.g. int/int
- **Prevent integer division**: force floating-point division
- **Match function parameter types:** satisfy function argument requirements

Example:

(float)a / b

(int)x

Type casting C

Risks

- Data loss (float to int): Fractional part permanently discarded e.g. float x=7.89; int y=(int)x; // y=7
- Overflow (large to small type): The value exceeds target type range e.g. int x=300; char y=(char)x; // overflow
- Reduced precision (double to float): There is loss of decimal accuracy e.g. double x=123.456789123; float y=(float)x; // precision reduced

Summary

- Type casting converts one data type to another
- Main use: control type behavior in expressions
- Two types: implicit (automatic) and explicit (manual)
- Must be used carefully to avoid data loss and overflow

Lecture 9: Variables , Constants and Typecasting in C

Summary

- The lecture introduced variables and constants in C, showing how to declare them correctly and apply appropriate naming conventions to improve program readability and maintainability.
- Different types of constants, including integer, floating-point, character, and string constants, were discussed, together with their uses in programs.
- The lesson also highlighted the differences between variables and constants, emphasizing their roles in storing and managing data during program execution.
- Finally, we learned about type conversion in C, including both implicit conversions performed automatically by the compiler and explicit conversions achieved through type casting.

Type casting C

Activity: In your breakout room:

- Copy the table below
- Examine each expression or scenario.
- Identify whether the casting is **implicit (automatic)** or **explicit (manual)**.
- Give a **reason** why the cast is used.
- Mention any **risk or consideration** (e.g., data loss, overflow, unexpected behavior).

SN	Expression / Scenario	Type of Casting (Implicit/Explicit)	Reason for Casting	Risk / Consideration
1	int x = 5; double y = x;			
2	double a = 3.7; int b = (int)a;			
3	int x = 300; char ch = (char)x;			
4	float f = 2.5; int i = f;			

Lecture 9: Variables , Constants and Typecasting in C

Exercise

a) Explain the following concepts with regard to C variables

i. Local variable

ii. Formal parameter

b) Explain why it is important to declare variables before use in C language

c) Write a simple program to convert amount of money in US dollars to KES. Assume 1$ is currently KES 147.00.

d) Write a program to read temperate in degree Celsius then convert to its equivalent degree Fahrenheit (Hint: F=9/5*C+32), where F and C denote Fahrenheit and Celsius respectively.

e) Explain the difference between implicit and explicit typecasting in C.

f) Explain why it is risky to cast a large integer value into a char type

g) Give one reason why programmers might use typecasting when performing arithmetic operations between different data types.

References

1. An introduction to programming by Wu Thomas, Norman and Theodore: McGrill (1997).
2. C programming language by Brian W. Kernighan, Dennis Ritchie, 2nd ed: prentice hall(2003).
3. C how to program by H.M. Deitel and P.J.Deitel, 3rd ed: Prentice hall(2001).